SIMATS ENGINEERING

Department of Information Technology

ITA05 – Computer Vision

COMPUTER VISION – ASSIGNMENT 2 (CO2)

OLD PHOTOGRAPH RESTORATION SYSTEM

CO2: Apply image enhancement and segmentation techniques to restore damaged photographs and identify meaningful image regions.

Assignment Details	Information	Assignment Details	Information
Student Name	STHUTI SHARMA S	Register Number	192572072
Department / Section	Information Technology	Academic Year / Semester	2025–2026
Faculty Name	DR. MURALI P	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Date of Submission		Date of Evaluation	

SCENARIO

A college archive has a collection of old family and institutional photographs that have become faded and damaged over time. The photographs show loss of contrast, yellow/sepia colour shifts, scratches, dust, stains, ink marks, creases and uneven fading. Some images are slightly blurred, while others contain small torn or missing regions. The archive wants an Old Photograph Restoration System that (1) improves brightness, contrast and visual clarity while preserving original facial features and fine details, and (2) identifies damaged regions such as scratches, stains, dust spots, creases and torn areas so that they can be repaired or reviewed. The restored result must retain the historical appearance of the photograph and must not invent or alter important details.

You are the vision engineer. For every task, three candidate techniques are provided. Implement all three techniques on the same set of old photographs, compare their performance, select the most suitable technique, and justify the selection with experimental evidence. Pay special attention to preserving faces, clothing, background details and original photographic tone. State and justify any additional assumptions.

OPERATING DATA

Parameter	Operating Data
Image resolution	A4, 2480 × 3508 pixels (300 DPI)
Image format	8-bit grayscale
Text stroke width	2–4 pixels
Contrast	Faded ink is 30–50 grey levels darker than paper
Lighting	Uneven illumination with one-sided shadows
Noise	Gaussian noise ($\sigma \approx 6$) and 1% salt-and-pepper noise
Scanner noise	Horizontal stripes every 6 pixel rows
Processing time	Maximum 3 seconds per page
Target accuracy	Text segmentation Dice score ≥ 0.90
Hardware	4 CPU cores, 16 GB RAM, no GPU
Software	Python, OpenCV and NumPy

TASK 1 – GREY LEVEL TRANSFORMATION ANALYSIS [4 Marks]

The old photographs contain faded areas and low contrast. Apply and compare three grey-level transformation techniques to recover visible details without losing highlights or shadow details.

Candidate Techniques

- A. Negative and Log Transformation
- B. Power-Law (Gamma) Transformation
- C. Piecewise-Linear Contrast Stretching

Questions

1. Implement all three transformations using Python/OpenCV on the same photographs.
2. Compare subject-to-background contrast, recovery of faint details, clipping effects and processing time.
3. Plot the transformation curves and corresponding input/output histograms.
4. Perform bit-plane slicing and identify which bit planes contain useful image details and which mainly contain noise or damage.
5. Analyse the effect of applying the same gamma value to both bright and dark photographs.
6. Compare a pretrained low-light or image-enhancement model with the selected transformation.

Evidence Expected: Transformation curves, before-and-after images, histograms, contrast comparison table and justified technique selection.

TASK 2 – HISTOGRAM PROCESSING AND ENHANCEMENT [4 Marks]

Old photographs have different exposure levels, faded regions and uneven tonal distribution. Apply and compare three histogram-based enhancement techniques.

Candidate Techniques

- A. Global Histogram Equalization
- B. Histogram Matching
- C. Adaptive Histogram Equalization (CLAHE)

Questions

7. Implement all three techniques using Python/OpenCV.
8. Compare detail visibility, contrast improvement, noise amplification and correction of faded or overexposed regions.
9. Generate histograms and Cumulative Distribution Function (CDF) plots.
10. Analyse why global histogram equalization may fail when a photograph contains both very dark and very bright regions.
11. Experiment with different CLAHE tile sizes and clip limits.
12. Examine the effect of applying global histogram equalization to a photograph in which a large area has a nearly uniform background.

Evidence Expected: Histograms, CDF plots, before-and-after images, contrast/entropy comparison and justified parameter selection.

TASK 3 – SPATIAL FILTERING AND IMAGE SMOOTHING [5 Marks]

Old photographs contain film grain, dust particles and salt-and-pepper-like damage. Apply three smoothing filters to reduce noise while preserving facial features, hair, clothing texture and fine edges.

Candidate Techniques

- A. Box (Averaging) Filter
- B. Gaussian Filter
- C. Median Filter

Questions

13. Implement all three filters using 3×3, 5×5 and 7×7 kernels.
14. Compare noise removal, detail preservation and processing time.
15. Calculate PSNR and SSIM for each filter.
16. Plot grayscale intensity profiles across selected facial edges, object boundaries or textured regions.
17. Demonstrate the difference between correlation and convolution using a 3×3 filter mask.
18. Analyse the consequences of using a large 7×7 filter on small details such as eyes, hair strands and narrow object boundaries.
19. Compare a classical filtering technique with a pretrained denoising model such as DnCNN.

Evidence Expected: Filtered images, PSNR/SSIM comparison table, intensity profiles and justified filter/kernel selection.

TASK 4 – IMAGE SHARPENING AND EDGE ENHANCEMENT [5 Marks]

Some old photographs are slightly blurred, making faces and object boundaries unclear. Apply three sharpening techniques to improve perceived detail without creating artificial edges.

Candidate Techniques

- A. Laplacian Sharpening
- B. Unsharp Masking / High-Boost Filtering
- C. Gradient-Based Sharpening

Questions

20. Implement all three sharpening techniques using Python/OpenCV.
21. Compare edge sharpness, noise amplification, halo effects and preservation of natural facial details.
22. Calculate the variance of Laplacian to measure image sharpness.
23. Plot intensity profiles across representative facial or object edges.
24. Explain why smoothing before sharpening may improve the output.
25. Analyse the effect of applying high-boost filtering with a large amplification factor to photographs containing scratches and grain.
26. Compare a traditional sharpening method with a pretrained deep-learning deblurring model.

Evidence Expected: Sharpened images, stroke profiles, sharpness/noise comparison and justified filter selection.

TASK 5 – FREQUENCY-DOMAIN LOW-PASS FILTERING [5 Marks]

Digitized old photographs may contain film grain, fine scratches and periodic scanning artefacts. Apply three frequency-domain low-pass filters to reduce unwanted high-frequency noise while preserving important image details.

Candidate Techniques

- A. Ideal Low-Pass Filter (ILPF)

- B. Butterworth Low-Pass Filter (BLPF)
- C. Gaussian Low-Pass Filter (GLPF)

Questions

27. Implement all three filters using Fourier Transform techniques.
28. Display the frequency spectrum and identify frequency patterns associated with grain, fine scratches or periodic scanning artefacts.
29. Compare the filters using different cut-off frequencies.
30. Measure ringing effects, loss of fine details, edge blurring and processing time.
31. Compare frequency-domain Gaussian filtering with spatial-domain Gaussian filtering in terms of detail preservation and noise reduction.
32. Analyse the effect of omitting padding before applying the Discrete Fourier Transform.
33. Evaluate edge/detail preservation before and after frequency-domain filtering using edge retention or a suitable image-quality measure.

Evidence Expected: Frequency spectra, filtered images, ringing/blur comparison and OCR performance table.

TASK 6 – FREQUENCY-DOMAIN HIGH-PASS FILTERING [4 Marks]

Old photographs may contain uneven fading, darkened corners and gradual illumination or colour changes. Apply three frequency-domain enhancement techniques to improve tonal consistency and reveal details.

Candidate Techniques

- A. Ideal / Butterworth High-Pass Filter
- B. Gaussian High-Pass Filtering with High-Frequency Emphasis
- C. Homomorphic Filtering

Questions

34. Implement all three techniques on shadow-affected photographs.
35. Compare detail sharpness, correction of uneven fading, noise amplification and preservation of overall photographic tone.
36. Measure local contrast and intensity variation in selected uniform regions.
37. Analyse the consequences of applying an ideal high-pass filter directly to an old photograph, especially around faces and strong edges.
38. Examine the effect of interchanging low-frequency and high-frequency gain parameters in homomorphic filtering.
39. Compare homomorphic filtering with a pretrained illumination-correction or low-light enhancement model.

Evidence Expected: High-pass filtered images, contrast/evenness comparison table and justified technique selection.

TASK 7 – POINT, LINE AND EDGE DETECTION [5 Marks]

The restoration system must locate scratches, dust spots, crease lines, stains and torn boundaries. Apply and compare three detection approaches.

Candidate Techniques

- A. Point and Line Detection Masks
- B. Gradient Edge Operators (Roberts, Prewitt and Sobel)
- C. Advanced Edge Detection (Marr–Hildreth and Canny)

Questions

40. Implement all three detection approaches using Python/OpenCV.
41. Detect dust spots, scratches, crease lines, stain boundaries and torn or missing regions.
42. Compare the techniques based on noise sensitivity, false detections and missed damage regions.
43. Calculate precision and recall using manually annotated ground-truth images.
44. Justify suitable thresholds for detecting small dust spots, thin scratches and major damaged boundaries.
45. Analyse why directional line masks may fail when scratches and creases occur at different orientations.
46. Compare a traditional edge detector with a deep-learning-based edge detector such as HED.

Evidence Expected: Detection maps, precision/recall comparison table and justified detector selection.

TASK 8 – IMAGE THRESHOLDING AND BINARIZATION [4 Marks]

The restoration system must separate damaged regions from relatively undamaged photograph areas. Fading, stains and uneven tones make this difficult. Apply and compare three thresholding techniques.

Candidate Techniques

- A. Basic Global Thresholding
- B. Otsu's Thresholding
- C. Adaptive Thresholding

Questions

47. Implement all three thresholding techniques using Python/OpenCV.
48. Compare their performance on lightly damaged, heavily faded and unevenly exposed photographs.
49. Generate binary damage masks and corresponding histograms.
50. Calculate Dice scores to evaluate damage-region segmentation accuracy.
51. Analyse why global thresholding may fail when damage and undamaged regions have overlapping intensity values.
52. Examine the effect of changing the block size and constant value in adaptive thresholding.
53. Compare traditional thresholding with a deep-learning-based image-damage segmentation model.

Evidence Expected: Binary images, annotated histograms, Dice score comparison and justified thresholding method.

TASK 9 – REGION-BASED SEGMENTATION AND INTEGRATED PIPELINE DESIGN [6 Marks]

The restoration system must identify connected damaged areas such as stains, scratches, creases and torn regions, while distinguishing the main subject from the background where needed. Apply three segmentation approaches and integrate the best techniques into a complete photograph-restoration system.

Candidate Techniques

- A. Region Growing
- B. Region Splitting and Merging
- C. Edge Linking and Boundary Detection

Questions

54. Implement all three segmentation techniques using Python/OpenCV.
55. Compare their ability to identify stains, ink marks, dust clusters, creases and torn or missing regions.

56. Calculate Intersection over Union (IoU) for damage-region detection and boundary localization error.
57. Develop an integrated photograph-restoration pipeline using the best techniques selected from Tasks 1–8.
58. Prepare a block diagram showing the complete processing sequence.
59. Measure the processing time of each stage and verify whether the complete system satisfies the 5-second-per-photo requirement.
60. Analyse the effect of selecting an incorrect seed point during region growing, particularly when a stain has a similar tone to the surrounding photograph.
61. Examine how overlapping scratches, creases and interrupted damage boundaries affect edge linking and region detection.
62. Compare traditional segmentation with a promptable model such as Segment Anything Model 2 (SAM 2) for identifying damaged regions.

Evidence Expected: Segmentation outputs, IoU comparison, corner localization results, integrated pipeline block diagram, stage-wise timing and final decision table.

TASK 10 – RESTORATION SYSTEM ADAPTATION FOR COLOUR AND HIGH-RESOLUTION PHOTOGRAPHS [8 Marks]

The archive now receives high-resolution colour photographs (4000 × 3000 pixels) of old prints, captured using mobile phones or flatbed scanners. Some images have strong sepia/yellow colour casts, faded faces, scratches and stains. A few photographs contain handwritten dates or notes on the reverse side that must not be erased. JPEG compression may introduce blocking artefacts. The restored image must be processed in less than 5 seconds using the same laboratory computer, and important original details must be preserved.

Candidate Techniques

- A. Retain – Use the existing restoration pipeline without modification.
- B. Modify – Resize images, retune enhancement and filtering parameters, and process luminance and colour channels separately to reduce colour casts while preserving natural skin tones.
- C. Redesign – Use colour restoration, frequency-domain damage reduction and inpainting for small damaged regions.

Questions

63. Implement all three strategies using Python/OpenCV on at least 15 old colour photographs.
64. Analyse the limitations of the existing pipeline when processing high-resolution, faded and damaged colour photographs.
65. Compare colour correction, detail enhancement, noise and damage removal, and preservation of facial and background details under the new conditions.
66. Measure damage-mask Dice score, colour fidelity, edge/detail preservation, perceptual quality and processing time. Where an undamaged reference is available, also report PSNR and SSIM.
67. Decide whether the existing pipeline should be RETAINED, MODIFIED or REDESIGNED.
68. Develop an improved pipeline that satisfies the restoration-quality and processing-time requirements.
69. Identify one operating condition under which the selected strategy may fail.
70. Explain how image restoration or inpainting models could support the improved system, and discuss the risk of generating details that were not present in the original photograph.

Evidence Expected: Comparison of three strategies, quantitative performance measurements, revised pipeline and justified RETAIN/MODIFY/REDESIGN decision.

MARK DISTRIBUTION

Task	Task Title	Marks

1	Grey Level Transformation Analysis	4
2	Histogram Processing and Enhancement	4
3	Spatial Filtering and Image Smoothing	5
4	Image Sharpening and Edge Enhancement	5
5	Frequency-Domain Low-Pass Filtering	5
6	Frequency-Domain High-Pass Filtering	4
7	Point, Line and Edge Detection	5
8	Image Thresholding and Binarization	4
9	Region-Based Segmentation and Integrated Pipeline Design	6
10	Restoration System Adaptation for Colour and High-Resolution Photographs	8
	TOTAL	50

COMMON SUBMISSION REQUIREMENTS

71. Python/OpenCV source code with comments.
72. Original and restored photograph images.
73. Results obtained by executing all three techniques for every task.
74. Comparison tables containing suitable performance metrics.
75. Graphs, histograms, intensity profiles and segmentation masks wherever applicable.
76. Advantages and limitations of each technique.
77. Final technique selection supported by experimental evidence.
78. An integrated photograph-restoration pipeline block diagram.
79. Processing-time measurements with units.
80. Final conclusion explaining the suitability of the selected techniques and how original details are preserved.

GENERAL INSTRUCTIONS

Use the same test photographs when comparing the three candidate techniques within a task. Clearly state the dataset size, preprocessing steps, parameter values and hardware/software environment. Do not report assumed or fabricated measurements as experimental results. If a pretrained model cannot be executed, clearly identify the comparison as a reasoned discussion rather than an experimental result. Do not invent facial features or historical details during restoration; clearly label any inpainting or generative reconstruction and retain the unedited original.

ABSTRACT

Old photographs are valuable historical records, but ageing, storage conditions, handling and digitization can introduce fading, low contrast, colour shifts, dust, scratches, stains, creases, blur and missing regions. This assignment develops a systematic computer-vision approach for analysing and restoring such photographs while preserving their original visual evidence. The work follows the ten tasks and candidate-technique comparisons specified in the Assignment 2 brief.

Problem and objective

The system has two linked objectives: improve brightness, contrast and visual clarity; and identify damaged regions for review or cautious repair. Restoration must preserve facial features, clothing texture, background objects and original photographic tone. It must not invent historically important details. The original image is therefore retained as an immutable reference, and all enhanced or repaired outputs are saved separately.

Methodology

Three candidate techniques are considered for each task. The report explains their mathematical basis, implementation approach, parameter choices, expected behaviour, advantages, limitations and evaluation metrics. The same test photographs must be used within each comparison. The assignment brief specifies Python, OpenCV and NumPy, an A4 grayscale reference of 2480 × 3508 pixels, and a CPU-only environment.

Evidence and integrity

The supplied brief requires executed outputs, plots, histograms, masks, comparison tables and measured processing time. No photographs, annotated masks or execution logs were supplied with the assignment prompt. Consequently, this report distinguishes theoretical comparison and implementation guidance from measured experimental outcomes. Numerical performance values are intentionally left as fields to be completed after running the code on the selected dataset; no fabricated PSNR, SSIM, Dice, IoU or timing values are presented.

Expected outcome

A conservative hybrid pipeline is proposed: luminance-aware enhancement, median or edge-preserving denoising, restrained sharpening, damage candidate detection, region segmentation, human review and optional non-generative inpainting for small confirmed defects. For high-resolution colour inputs, the preferred direction is to modify the pipeline by separating luminance and chrominance processing and using scale-aware parameters.

Important: This document is a detailed assignment report and implementation guide. Insert actual output images and measured results from your own execution before submission.

TABLE OF CONTENTS AND REPORT ORGANIZATION

The original seven-page assignment brief is reproduced at the beginning of this document. The following report expands the ten tasks in the same order, retaining the candidate methods, questions and evidence expectations. The page numbers below refer to the report section sequence after the reproduced brief and may be refreshed in Word after editing.

Front matter

Cover page; abstract; report organization; problem definition; system requirements; experimental protocol.

Tasks 1–2

Grey-level transformation analysis; histogram processing and enhancement.

Tasks 3–4

Spatial filtering and smoothing; image sharpening and edge enhancement.

Tasks 5–6

Frequency-domain low-pass filtering; frequency-domain high-pass filtering.

Tasks 7–8

Point, line and edge detection; image thresholding and binarization.

Tasks 9–10

Region-based segmentation and integrated pipeline design; adaptation for colour and high-resolution photographs.

Closing sections

Overall conclusion; limitations and future work; implementation appendix; result-recording templates and references.

- Task 1 – Grey Level Transformation Analysis (4 marks)
- Task 2 – Histogram Processing and Enhancement (4 marks)
- Task 3 – Spatial Filtering and Image Smoothing (5 marks)
- Task 4 – Image Sharpening and Edge Enhancement (5 marks)
- Task 5 – Frequency-Domain Low-Pass Filtering (5 marks)
- Task 6 – Frequency-Domain High-Pass Filtering (4 marks)
- Task 7 – Point, Line and Edge Detection (5 marks)
- Task 8 – Image Thresholding and Binarization (4 marks)
- Task 9 – Region-Based Segmentation and Integrated Pipeline Design (6 marks)
- Task 10 – Restoration System Adaptation for Colour and High-Resolution Photographs (8 marks)

1. INTRODUCTION AND PROBLEM DEFINITION

Photographic archives contain images whose information value is much greater than their current visual quality. A faded portrait may still contain important identity, clothing, architectural and documentary details. Restoration is therefore not simply a matter of making an image look attractive. It is a constrained image-processing problem in which visibility is improved without changing the evidence represented by the photograph.

1.1 Common forms of degradation

Photographs may exhibit reduced global contrast, non-uniform illumination, yellow or sepia shifts, film grain, scanner stripes, isolated dust, elongated scratches, stains, folds, creases, blur and torn or missing areas. Several forms can occur together. A dark stain may overlap a face, a scratch may cross a textured garment, or a crease may create a strong edge that resembles a genuine object boundary.

1.2 Restoration versus reconstruction

Enhancement modifies the appearance of existing pixels to make information easier to inspect. Restoration attempts to reverse a degradation process. Inpainting estimates pixels in a damaged region from surrounding information. These operations have different evidential status: enhancement can reveal existing content, while inpainting may synthesize a plausible replacement. Any reconstructed region must be clearly identified and the unedited original preserved.

1.3 Main engineering challenge

The system must balance noise removal and detail preservation. Strong smoothing removes grain but can erase eyes, hair strands and narrow boundaries. Strong sharpening improves edge contrast but also amplifies scratches and noise. Global contrast enhancement may improve a faded region while clipping highlights or exaggerating stains. Segmentation must separate damage from authentic photographic content.

- Maintain a read-only copy of every source photograph.
- Use identical images for the three candidate techniques within each task.
- Record preprocessing, parameters, software versions and hardware.
- Keep original, enhanced, damage-mask and repaired outputs as separate files.
- Report measured results only when they have actually been computed.

2. OPERATING REQUIREMENTS AND ASSUMPTIONS

The operating data in the supplied assignment brief defines the baseline conditions. These values should guide the experiment design and should not be silently replaced by easier settings.

2.1 Explicit assumptions

The report assumes that each source image is available as a lossless or minimally compressed scan, that the original is not overwritten, and that damage annotations can be created manually for evaluation. If the dataset differs from the specified A4 grayscale baseline, the difference must be recorded. The 5-second-per-photo condition in Task 9 and Task 10 is treated as a separate system-level target from the 3-second-per-page baseline operating data.

2.2 Test set

Select representative photographs covering light fading, heavy fading, uneven shadow, Gaussian noise, impulse noise, periodic stripes, scratches, stains, creases and missing corners. Use the same selected images when comparing candidates in a task. Record image identifiers and avoid selecting only images that favour one technique.

Parameter	Specified requirement	Design implication
Reference image	A4, 2480 × 3508 pixels, 300 DPI	Use a consistent image size for baseline comparison.
Image type	8-bit grayscale	Use uint8 grayscale for Tasks 1–9 unless a task explicitly requires colour.
Text / thin stroke	2–4 pixels	Avoid filters that erase very narrow structures.
Faded ink contrast	30–50 grey levels darker than paper	Threshold selection must account for modest foreground-background separation.
Illumination	Uneven, one-sided shadows	Prefer local or illumination-aware enhancement.
Noise	Gaussian $\sigma \approx 6$; 1% salt-and-pepper	Evaluate both Gaussian and impulse-noise handling.
Scanner artefact	Horizontal stripes every 6 pixel rows	Inspect periodic frequency peaks and row-wise intensity.
Time	Maximum 3 seconds per page in baseline data	Measure complete processing on the stated CPU environment.
Target	Text segmentation Dice ≥ 0.90	Use manually annotated ground truth and consistent mask convention.
Hardware/software	4 CPU cores, 16 GB RAM, no GPU; Python/OpenCV/NumPy	Prefer efficient classical methods for the baseline.

3. EXPERIMENTAL DESIGN AND EVALUATION PROTOCOL

A fair comparison requires controlled inputs, repeatable preprocessing and a consistent evaluation method. A technique should not be selected only because one output looks more dramatic. The decision must consider visual fidelity, damage detection, runtime and preservation of original details.

3.1 Dataset preparation

Create a dataset folder with separate subfolders for originals, annotations, outputs and reports. Give each image a unique identifier. Record dimensions, bit depth, file format and any known degradation. For each damage category, create a binary ground-truth mask: foreground pixels indicate confirmed damage and background pixels indicate areas not labelled as damage. Store annotations without modifying source images.

3.2 Preprocessing

Load images with OpenCV, validate that the file opened successfully, and convert to grayscale where required. Do not resize the baseline images unless a task explicitly tests resizing. If resizing is necessary, document interpolation and scale. Keep image data in uint8 for standard OpenCV operations; convert to float32 for Fourier operations and normalize carefully.

3.3 Quantitative measures

Use PSNR and SSIM when a clean reference exists. Use Dice and IoU for segmentation against manually labelled masks. Precision and recall describe false alarms and missed damage. Variance of Laplacian is a sharpness indicator, not a complete quality score. Entropy describes intensity distribution but does not necessarily mean perceptual quality. Runtime should be measured after a warm-up and repeated several times.

3.4 Visual inspection

Inspect the same crop locations for faces, hair, clothing texture, background, highlights and shadows. Compare full-frame images and enlarged crops. Look for halos, ringing, clipped tones, over-smoothed regions, false damage detections and colour shifts. Maintain a side-by-side contact sheet for each candidate.

Metric	Formula / interpretation	Use
PSNR	$10 \log_{10}(MAX^2 / MSE)$	Higher is better when a valid clean reference exists.
SSIM	Structural similarity of luminance, contrast and structure	Closer to 1 indicates greater similarity.
Dice	$2\|A \cap B\| / (\|A\|+\|B\|)$	Measures overlap of predicted and ground-truth damage masks.
IoU	$\|A \cap B\| / \|A \cup B\|$	Measures region overlap.
Precision	TP / (TP+FP)	Higher precision means fewer false damage alarms.
Recall	TP / (TP+FN)	Higher recall means fewer missed damage pixels.

TASK 1 – GREY LEVEL TRANSFORMATION ANALYSIS – CANDIDATE METHODS AND MATHEMATICAL BASIS (1/5)

Grey-level transformations map an input intensity r to an output intensity s. For an 8-bit image, r and s lie in the range 0–255. Negative transformation uses s = 255 − r and reverses brightness ordering; it can expose details in selected tonal ranges but usually does not provide a natural restoration. Log transformation uses s = c log(1+r), compressing a wide dynamic range and expanding darker values. Power-law transformation uses s = c r^γ after normalization to [0,1]. Gamma below one brightens midtones, while gamma above one darkens them. Piecewise-linear contrast stretching maps selected input ranges to a wider output range and can be tuned to protect highlights and shadows.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 1 – GREY LEVEL TRANSFORMATION ANALYSIS – IMPLEMENTATION PLAN (2/5)

Apply all transformations to the same grayscale images. Normalize intensities before logarithmic or power-law operations and convert back to uint8 only after clipping. For a fair comparison, record the exact gamma value, log scale factor and piecewise control points. A robust piecewise stretch can use low and high percentiles rather than the absolute minimum and maximum, because isolated dust pixels can distort extrema. Plot each transformation curve on common axes and generate input/output histograms.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 1 – GREY LEVEL TRANSFORMATION ANALYSIS – COMPARISON AND EXPECTED BEHAVIOUR (3/5)

Negative transformation is useful as an inspection view but is not normally the best final restoration because it reverses the photograph's tonal meaning. Log transformation may reveal shadow detail but can compress highlight differences. Gamma correction is simple and efficient, yet a single gamma value may brighten already-bright areas too much or darken an image that needs lifting. Piecewise stretching offers explicit control over faded midtones and clipping, but poorly selected breakpoints can flatten important regions.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 1 – GREY LEVEL TRANSFORMATION ANALYSIS – BIT-PLANE SLICING (4/5)

An 8-bit grayscale image contains eight bit planes, from the least significant bit (bit 0) to the most significant bit (bit 7). High-order planes usually carry coarse structure and broad tonal information. Low-order planes may contain fine texture, scanner noise, grain and quantization variation. Display each plane as a binary image. A plane should not be labelled as noise solely because it appears speckled: fine photographic texture may also be represented there. Compare the planes with known damage and detail regions.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 1 – GREY LEVEL TRANSFORMATION ANALYSIS – PRETRAINED ENHANCEMENT COMPARISON (5/5)

A pretrained low-light or enhancement model can learn complex nonlinear corrections, but its output may change local texture or introduce plausible details. If the model cannot be executed on the CPU-only laboratory computer, discuss it as a reasoned comparison, not as an experiment. Record model name, version, input constraints, runtime and any resizing if it is tested. The classical transformation remains easier to explain, reproduce and audit.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 2 – HISTOGRAM PROCESSING AND ENHANCEMENT – HISTOGRAM AND CUMULATIVE DISTRIBUTION (1/5)

A grayscale histogram counts pixels at each intensity. The cumulative distribution function (CDF) is the cumulative sum of the normalized histogram. Global histogram equalization maps intensities using the CDF, attempting to spread frequently occurring values across the available range. It is computationally efficient and can improve global contrast when the image has a narrow intensity distribution. However, it does not understand semantic regions and may exaggerate stains, grain or scanner marks.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 2 – HISTOGRAM PROCESSING AND ENHANCEMENT – HISTOGRAM MATCHING (2/5)

Histogram matching transforms the source intensity distribution toward a reference distribution. Select a reference photograph with a similar historical tone and exposure objective; do not choose an unrelated modern image simply because its histogram is broad. Compute source and reference CDFs, then map each source intensity to the reference intensity with the closest cumulative probability. Matching can make a set of archive images more consistent, but an unsuitable reference can impose an unnatural tone.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 2 – HISTOGRAM PROCESSING AND ENHANCEMENT – CLAHE AND PARAMETER SELECTION (3/5)

Contrast Limited Adaptive Histogram Equalization divides the image into tiles, equalizes local histograms and limits contrast amplification by clipping histogram peaks. Tile size controls the spatial scale of adaptation. Smaller tiles can reveal local details but may create a patchy appearance and amplify noise. Larger tiles approach global processing. A higher clip limit increases contrast and noise amplification; a lower limit is more conservative. Test multiple tile-grid sizes and clip limits on identical images.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 2 – HISTOGRAM PROCESSING AND ENHANCEMENT – FAILURE ANALYSIS AND LARGE UNIFORM BACKGROUNDS (4/5)

Global equalization may fail when a photograph contains both very dark and very bright areas because the global histogram combines both populations. A large nearly uniform background can dominate the histogram and cause the smaller subject region to receive an unsuitable mapping. Local equalization may help, but it can also create tile boundaries or exaggerate grain. Use local contrast, entropy, clipping percentage and visual inspection together; entropy alone is not a quality score.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 2 – HISTOGRAM PROCESSING AND ENHANCEMENT – SELECTION AND PRETRAINED COMPARISON (5/5)

For unevenly exposed photographs, CLAHE is a reasonable candidate because it adapts locally while limiting amplification. Its parameters must be selected conservatively and validated on faces and textured regions. Histogram matching is preferable when consistent tonal appearance across a collection is important and a trustworthy reference image exists. A pretrained enhancement model may provide stronger local adaptation but requires careful checking for texture hallucination and colour or tone shifts.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 3 – SPATIAL FILTERING AND IMAGE SMOOTHING – SPATIAL FILTERING FUNDAMENTALS (1/5)

Spatial filters compute an output pixel from a neighbourhood around the corresponding input pixel. A 3×3 kernel uses nine neighbouring values; 5×5 and 7×7 kernels cover progressively larger areas. Box filtering averages the neighbourhood uniformly. Gaussian filtering applies larger weights near the centre and smaller weights farther away. Median filtering sorts neighbourhood values and selects the middle value, making it effective against isolated salt-and-pepper impulses.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 3 – SPATIAL FILTERING AND IMAGE SMOOTHING – IMPLEMENTATION AND KERNEL EXPERIMENT (2/5)

Apply box, Gaussian and median filters with 3×3, 5×5 and 7×7 windows. Keep border handling consistent and record the OpenCV border mode. For Gaussian filtering, record sigma or allow OpenCV to derive it from the kernel size, but use the same rule for all test images. Median filtering requires an odd kernel size. Use identical crops around eyes, hair, clothing seams and object boundaries when comparing outputs.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 3 – SPATIAL FILTERING AND IMAGE SMOOTHING – QUALITY MEASURES AND PROFILES (3/5)

If a clean reference image exists, calculate PSNR and SSIM for each filter. If no clean reference exists, do not invent these scores; use annotated crops, edge retention and expert inspection instead. Plot intensity profiles across representative facial edges or narrow details. A good filter should suppress isolated noise without flattening a genuine edge or removing small features. Report processing time in milliseconds and state image dimensions and hardware.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 3 – SPATIAL FILTERING AND IMAGE SMOOTHING – CORRELATION AND CONVOLUTION (4/5)

Correlation slides a kernel over an image without flipping the kernel. Convolution flips the kernel horizontally and vertically before sliding it. For symmetric masks such as a Gaussian kernel, correlation and convolution produce the same result. For an asymmetric directional mask, the result can differ because the orientation is reversed. Demonstrate the distinction with a non-symmetric 3×3 mask and a small test matrix; show the flipped kernel explicitly.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 3 – SPATIAL FILTERING AND IMAGE SMOOTHING – TRADE-OFFS AND SELECTION (5/5)

A 7×7 filter can reduce more noise but may blur eyes, hair strands and narrow boundaries. Box filtering can create a flat, softened appearance. Gaussian filtering provides smoother, spatially weighted averaging but does not remove impulse noise as effectively as a median filter. Median filtering is the preferred classical choice for isolated salt-and-pepper spots, while a modest Gaussian filter may be more suitable for approximately Gaussian grain. A combined, mild approach should be validated rather than applied automatically.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 4 – IMAGE SHARPENING AND EDGE ENHANCEMENT – SHARPENING PRINCIPLES (1/5)

Sharpening increases local contrast around transitions. Laplacian sharpening uses a second-order derivative to highlight rapid intensity changes. Unsharp masking blurs the image, subtracts the blurred version from the original to obtain a detail mask, and adds a scaled mask back. High-boost filtering generalizes this by using an amplification factor. Gradient-based sharpening uses first-order derivatives such as Sobel to emphasize edge direction and magnitude.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 4 – IMAGE SHARPENING AND EDGE ENHANCEMENT – IMPLEMENTATION OF CANDIDATE METHODS (2/5)

For Laplacian sharpening, compute the Laplacian in a signed or floating-point representation and combine it with the original image using a documented sign convention. For unsharp masking, generate a Gaussian-blurred image, calculate the difference mask and add a controlled fraction of that mask. For gradient-based sharpening, compute horizontal and vertical derivatives, combine their magnitude and add a restrained edge component. Clip only at the final conversion step.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 4 – IMAGE SHARPENING AND EDGE ENHANCEMENT – COMPARISON AND SHARPNESS MEASUREMENT (3/5)

Compare perceived edge sharpness, amplification of grain and scratches, halo formation and preservation of facial detail. Variance of Laplacian can provide a simple sharpness indicator: a higher value often corresponds to more high-frequency content, but noise can also increase it. Therefore, interpret this metric alongside noise level and visual crops. Plot intensity profiles across the same edges before and after sharpening to reveal overshoot or halos.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 4 – IMAGE SHARPENING AND EDGE ENHANCEMENT – WHY SMOOTHING MAY PRECEDE SHARPENING (4/5)

Mild denoising before sharpening can reduce the amount of noise amplified by the sharpening operator. The denoising step must be conservative because smoothing can remove details that sharpening cannot recover. A sensible sequence is to remove obvious impulse noise, apply mild Gaussian or edge-preserving smoothing if necessary, then sharpen at a low strength. The sequence and parameters must be compared against a no-pre-smoothing baseline.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 4 – IMAGE SHARPENING AND EDGE ENHANCEMENT – HIGH-BOOST RISK AND DEEP DEBLURRING (5/5)

A large high-boost factor exaggerates both genuine edges and unwanted scratches, grain and compression artefacts. It may create bright and dark halos near strong boundaries and make faces appear artificial. A pretrained deblurring model may handle complex blur better than a simple filter, but can generate details not supported by the input. Use it only as a separately labelled comparison and preserve the original image for verification.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 5 – FREQUENCY-DOMAIN LOW-PASS FILTERING – FOURIER-DOMAIN REPRESENTATION (1/5)

The two-dimensional Discrete Fourier Transform represents an image as spatial frequencies. Low frequencies describe broad illumination and slowly varying structures; high frequencies contain fine edges, texture, grain and some damage. The magnitude spectrum is commonly shifted so that low frequencies appear near the centre. Frequency-domain filtering multiplies the Fourier transform by a transfer function and then applies the inverse transform.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 5 – FREQUENCY-DOMAIN LOW-PASS FILTERING – IDEAL, BUTTERWORTH AND GAUSSIAN FILTERS (2/5)

An Ideal Low-Pass Filter passes frequencies inside a circular cutoff radius and rejects frequencies outside it. Its abrupt transition can produce ringing in the spatial image. A Butterworth Low-Pass Filter provides a smoother transition controlled by cutoff frequency and filter order. A Gaussian Low-Pass Filter has a smooth frequency response and generally avoids the pronounced ringing associated with an ideal cutoff. All three reduce high-frequency content, but may also remove fine photographic details.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 5 – FREQUENCY-DOMAIN LOW-PASS FILTERING – IMPLEMENTATION AND CUTOFF EXPERIMENT (3/5)

Convert the image to float32, compute the two-dimensional Fourier transform, shift the zero-frequency component to the centre, build the chosen transfer mask and multiply it with the shifted spectrum. Inverse-shift and inverse-transform, then normalize carefully for display. Test several cutoff frequencies using the same image. Record padding strategy, mask parameters, runtime and image dimensions. Avoid treating display normalization as a change to the underlying filtering operation.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 5 – FREQUENCY-DOMAIN LOW-PASS FILTERING – FREQUENCY ARTEFACTS AND PADDING (4/5)

Periodic horizontal scanner stripes can produce structured peaks or bands in the frequency spectrum, depending on their periodicity and orientation. Grain tends to occupy a broader high-frequency range, while thin scratches create directional energy. Padding reduces boundary discontinuities and helps prevent circular wraparound effects from contaminating opposite edges. Without padding, the DFT assumes periodic repetition; discontinuities at image boundaries can create spectral leakage and visible artefacts.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 5 – FREQUENCY-DOMAIN LOW-PASS FILTERING – EVALUATION AND SPATIAL GAUSSIAN COMPARISON (5/5)

Measure edge retention, fine-detail loss, ringing and blur. Compare frequency-domain Gaussian filtering with spatial Gaussian filtering using equivalent effective smoothing scales. For ideal infinite-support Gaussian kernels, the Fourier relationship is exact, but practical discrete kernels, truncation, padding and boundary handling cause differences. The most suitable low-pass filter is the one that reduces unwanted noise while retaining important details, not necessarily the one producing the smoothest image.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 6 – FREQUENCY-DOMAIN HIGH-PASS FILTERING – HIGH-PASS FILTERING AND ILLUMINATION (1/5)

High-pass filters suppress slowly varying components and retain rapid intensity changes. They can emphasize edges and local details, but a direct high-pass result often resembles an edge map rather than a natural photograph. Ideal and Butterworth high-pass filters complement their low-pass counterparts. Gaussian high-pass filtering uses a smooth transition. High-frequency emphasis combines a baseline gain with a scaled high-frequency component to retain more overall image appearance.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 6 – FREQUENCY-DOMAIN HIGH-PASS FILTERING – HOMOMORPHIC FILTERING (2/5)

Homomorphic filtering models an image as the product of illumination and reflectance. Taking the logarithm converts the product into a sum. A frequency-domain filter can then attenuate low-frequency illumination variation and enhance reflectance-related detail. After inverse transformation, exponentiation returns to an image-like intensity representation. Parameters commonly include low-frequency gain, high-frequency gain and a transition cutoff. Their values determine how strongly shading is reduced and detail is emphasized.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 6 – FREQUENCY-DOMAIN HIGH-PASS FILTERING – PARAMETER INTERCHANGE AND FAILURE ANALYSIS (3/5)

If low- and high-frequency gain parameters are interchanged, the filter may suppress the desired illumination component while boosting broad shading, or may amplify noise and fine scratches. The effect should be demonstrated with a controlled parameter table. Applying an ideal high-pass filter directly to a photograph can create halos or ringing around faces and strong edges, discard broad tonal information and exaggerate small defects. It should not be treated as a complete restoration output.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 6 – FREQUENCY-DOMAIN HIGH-PASS FILTERING – EVALUATION AND MODEL COMPARISON (4/5)

Measure local contrast and intensity variation in selected regions that should be approximately uniform. Compare face regions, background shadows and known faded areas. Record whether local contrast improves without changing the overall historical tone. A pretrained illumination-correction model may provide learned adaptation, but its output must be checked for unnatural skin tones and altered texture. If not executed, describe it as a conceptual comparison only.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 6 – FREQUENCY-DOMAIN HIGH-PASS FILTERING – TECHNIQUE SELECTION (5/5)

Homomorphic filtering is the most principled of the listed methods for uneven illumination because it explicitly separates illumination and reflectance in a simplified model. It still depends on assumptions that may not hold for complex photographs. Use conservative gains, compare against a CLAHE baseline and retain the unprocessed image. High-pass outputs should be used as enhancement components or diagnostic maps, not blindly substituted for the original photograph.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 7 – POINT, LINE AND EDGE DETECTION – DAMAGE STRUCTURES AND DETECTION OBJECTIVE (1/5)

The system must identify isolated dust spots, elongated scratches, crease lines, stain boundaries and torn or missing regions. These structures differ in geometry and contrast. Dust may appear as a compact bright or dark point; a scratch may be thin and directional; a crease may be long but irregular; a stain may have a diffuse boundary; and a tear may create a discontinuity at the image edge. No single detector is expected to identify every category reliably.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 7 – POINT, LINE AND EDGE DETECTION – POINT AND LINE MASKS (2/5)

Point detection uses a mask that responds to a local intensity difference from the neighbourhood. Line masks are directional and can be designed for horizontal, vertical and diagonal structures. A scratch at an orientation not represented by the masks may produce a weak response. Use multiple orientations or rotate the mask bank. Threshold the response carefully and combine detections only after checking false positives from hair, clothing seams and genuine architectural lines.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 7 – POINT, LINE AND EDGE DETECTION – GRADIENT OPERATORS (3/5)

Roberts uses small diagonal difference operators and is sensitive to noise. Prewitt uses simple horizontal and vertical derivative masks. Sobel adds greater weighting to central pixels and is commonly used to estimate image gradients. Gradient magnitude can be calculated from horizontal and vertical responses. These operators detect intensity transitions but do not know whether an edge represents damage or authentic content.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 7 – POINT, LINE AND EDGE DETECTION – MARR–HILDRETH AND CANNY (4/5)

Marr–Hildreth detection typically smooths the image, applies a Laplacian of Gaussian and identifies zero crossings. It can detect edges at a chosen scale, but its response depends on smoothing and threshold parameters. Canny uses smoothing, gradient estimation, non-maximum suppression and hysteresis thresholding. It often produces thin connected edges, but thresholds must be adjusted to avoid responding to grain or missing faint scratches.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 7 – POINT, LINE AND EDGE DETECTION – METRICS AND DEEP DETECTOR COMPARISON (5/5)

Use manually annotated ground-truth damage masks to calculate precision and recall. Evaluate dust, scratches, creases and tears separately because a detector may perform differently across categories. Select thresholds based on a validation subset, then report results on a held-out subset where possible. A deep-learning detector such as HED can learn richer edge cues but requires model weights, runtime support and representative training data; do not claim experimental superiority without running it.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 8 – IMAGE THRESHOLDING AND BINARIZATION – THRESHOLDING OBJECTIVE (1/5)

Thresholding converts grayscale intensities into a binary classification. In this assignment, the binary output is used to propose damaged pixels or regions for further review. A binary mask should not be interpreted as proof that every selected pixel is damage. Faded photographic texture, hair, dark clothing and shadows may have intensities similar to stains or scratches. Thresholding is therefore a candidate-generation stage in a larger pipeline.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 8 – IMAGE THRESHOLDING AND BINARIZATION – GLOBAL THRESHOLDING (2/5)

Basic global thresholding compares every pixel with a single threshold T. Depending on the selected binary convention, values below T or above T become foreground. The threshold can be chosen manually from image statistics or a representative sample. This method is fast and easy to explain, but it assumes a reasonably separable intensity distribution. It may fail under one-sided shadows or when damage and undamaged regions overlap in intensity.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 8 – IMAGE THRESHOLDING AND BINARIZATION – OTSU THRESHOLDING (3/5)

Otsu's method selects a threshold that maximizes between-class variance, effectively seeking a split between two intensity populations. It is automatic and works best when the histogram contains reasonably distinct classes. A faded photograph with a large uniform background or multiple overlapping tonal populations may violate this assumption. Otsu's threshold should be compared with manual and adaptive approaches rather than assumed to be universally optimal.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 8 – IMAGE THRESHOLDING AND BINARIZATION – ADAPTIVE THRESHOLDING AND PARAMETERS (4/5)

Adaptive thresholding calculates a local threshold around each pixel. In OpenCV, the block size controls the neighbourhood dimensions and must be odd and greater than one. The constant C is subtracted from the local statistic. Smaller blocks adapt to small local changes but can amplify texture and noise; larger blocks respond to broader illumination changes. Changing C shifts the foreground decision and should be tested systematically.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 8 – IMAGE THRESHOLDING AND BINARIZATION – DICE EVALUATION AND MODEL COMPARISON (5/5)

For binary damage segmentation, compare the predicted mask with a manually annotated ground-truth mask. Dice is twice the intersection divided by the total foreground count in both masks. State clearly whether white or black represents damage. Evaluate lightly damaged, heavily faded and unevenly exposed photographs separately. A deep segmentation model may exploit texture and context, but requires suitable labelled examples and should be evaluated using the same masks and metrics.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 9 – REGION-BASED SEGMENTATION AND INTEGRATED PIPELINE DESIGN – REGION SEGMENTATION OBJECTIVE (1/5)

Region-based methods group pixels into connected or similar areas. They can turn scattered pixel responses into meaningful damage candidates such as a stain, ink mark, dust cluster, crease or torn region. Region segmentation should preserve the distinction between a candidate region and a confirmed damaged region. The system should permit a reviewer to accept, reject or adjust proposed masks, especially where damage overlaps a face or important object.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 9 – REGION-BASED SEGMENTATION AND INTEGRATED PIPELINE DESIGN – REGION GROWING (2/5)

Region growing begins with one or more seed pixels and adds neighbouring pixels that satisfy a similarity rule. Similarity may be based on intensity difference, colour distance or texture. Connectivity can be four-neighbour or eight-neighbour. A seed placed inside a stain may grow across the stain if the boundary contrast is sufficient. An incorrect seed or a loose threshold may leak into surrounding photographic content; a strict threshold may fragment one damaged region.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 9 – REGION-BASED SEGMENTATION AND INTEGRATED PIPELINE DESIGN – REGION SPLITTING AND MERGING (3/5)

Region splitting begins with a large region and recursively divides it when a homogeneity criterion is not satisfied. Merging combines adjacent regions whose properties are sufficiently similar. A quadtree is a common conceptual structure for rectangular splitting. The method can handle broad variations but may create block boundaries and may not align naturally with thin scratches. The homogeneity criterion, minimum region size and merge threshold must be documented.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 9 – REGION-BASED SEGMENTATION AND INTEGRATED PIPELINE DESIGN – EDGE LINKING AND BOUNDARY DETECTION (4/5)

Edge linking connects edge pixels into contours using proximity, orientation and continuity. It is useful when a tear or stain has a reasonably visible boundary. Broken edges, overlapping scratches and creases can interrupt contours or create false connections. Linking should use gap limits and direction constraints, then validate closed contours before filling them as regions. A boundary is not automatically a damage boundary; context and reviewer confirmation remain important.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 9 – REGION-BASED SEGMENTATION AND INTEGRATED PIPELINE DESIGN – IOU, BOUNDARY ERROR AND INTEGRATION (5/5)

Calculate IoU between predicted and ground-truth damage regions. Boundary localization error can be measured as an average or maximum distance between predicted and annotated boundaries, with the distance definition stated. The integrated pipeline should combine enhancement, mild denoising, damage candidate detection, thresholding, region segmentation, mask refinement and optional repair. Record stage-wise runtime and ensure the total meets the system requirement on the stated hardware.

Practical implementation notes

Use OpenCV and NumPy operations with explicit input validation, data-type handling and parameter logging. Save every candidate output using a filename that includes the image identifier, method and parameter set. Use identical input images and comparable output scaling. Keep a record of software versions and run conditions so that the comparison can be repeated.

Evidence to include in the final submission

Insert the original image, the output from each candidate method, an enlarged crop of a representative detail and the relevant plot or mask. Add a concise caption that states the method and parameters. If the output has not been executed, label the figure area as pending rather than presenting a simulated or illustrative result as measured evidence.

Comparison field	Method A	Method B	Method C
Visual detail / artefacts	Record after execution	Record after execution	Record after execution
Noise / false response	Record after execution	Record after execution	Record after execution
Metric value	Enter measured value	Enter measured value	Enter measured value
Runtime (ms)	Enter measured value	Enter measured value	Enter measured value

Results table intentionally contains measurement fields. Replace them with values obtained from the same test images; do not estimate or fabricate results.

TASK 10 – RESTORATION SYSTEM ADAPTATION (1/6): STRATEGY A – RETAIN THE EXISTING PIPELINE

The RETAIN strategy applies the existing grayscale restoration pipeline to the incoming colour photographs with minimal modification. It is the simplest option operationally because it reuses known stages and avoids a major redesign. However, the original pipeline was defined around 8-bit grayscale A4 images and the new scenario contains 4000 × 3000 colour photographs, strong sepia or yellow casts, JPEG blocking artefacts and handwritten reverse-side notes. Directly converting every image to grayscale discards chromatic information and may make natural skin-tone preservation impossible. The strategy should therefore be treated as a baseline rather than presumed suitable.

Required implementation and evidence

Implement RETAIN, MODIFY and REDESIGN on the same minimum set of 15 colour photographs. For each strategy, save a full-resolution result and a consistent enlarged crop. Record colour correction, detail enhancement, noise reduction, damage removal and preservation of faces and background. Where no clean reference exists, do not report reference-based PSNR or SSIM.

Decision record

Complete the final decision table after executing the strategies. The choice must be supported by observed outputs and measurements. This report recommends MODIFY on engineering grounds, but the final submitted decision should be checked against the actual results obtained on the 15-image dataset.

Evaluation item	RETAIN	MODIFY	REDESIGN
Colour fidelity	Enter result	Enter result	Enter result
Damage-mask Dice	Enter result	Enter result	Enter result
Detail preservation	Enter result	Enter result	Enter result
Per-photo runtime	Enter result	Enter result	Enter result
Operational complexity	Low	Moderate	High

At least 15 images are required by the brief. Replace each entry with measured evidence and state the evaluation method.

TASK 10 – RESTORATION SYSTEM ADAPTATION (2/6): STRATEGY B – MODIFY FOR COLOUR AND RESOLUTION

The MODIFY strategy retains the proven classical stages but adapts the data path. First, inspect image dimensions and colour space. Process luminance separately from chrominance so that contrast enhancement and sharpening do not unnecessarily alter colour. In an OpenCV implementation, convert BGR to a suitable luminance/chrominance representation such as LAB or YCrCb, enhance the luminance channel conservatively, and apply restrained chroma correction. Use scale-aware kernel sizes and consider processing a working-resolution copy for detection while preserving the original-resolution image for final output. Notes on the reverse side must be treated as documentary content, not damage.

Required implementation and evidence

Implement RETAIN, MODIFY and REDESIGN on the same minimum set of 15 colour photographs. For each strategy, save a full-resolution result and a consistent enlarged crop. Record colour correction, detail enhancement, noise reduction, damage removal and preservation of faces and background. Where no clean reference exists, do not report reference-based PSNR or SSIM.

Decision record

Complete the final decision table after executing the strategies. The choice must be supported by observed outputs and measurements. This report recommends MODIFY on engineering grounds, but the final submitted decision should be checked against the actual results obtained on the 15-image dataset.

Evaluation item	RETAIN	MODIFY	REDESIGN
Colour fidelity	Enter result	Enter result	Enter result
Damage-mask Dice	Enter result	Enter result	Enter result
Detail preservation	Enter result	Enter result	Enter result
Per-photo runtime	Enter result	Enter result	Enter result
Operational complexity	Low	Moderate	High

At least 15 images are required by the brief. Replace each entry with measured evidence and state the evaluation method.

TASK 10 – RESTORATION SYSTEM ADAPTATION (3/6): STRATEGY C – REDESIGN WITH COLOUR RESTORATION AND INPAINTING

The REDESIGN strategy introduces more substantial changes: colour-cast correction, frequency-domain reduction of periodic artefacts, damage segmentation and inpainting of small confirmed defects. It may provide greater flexibility for complex inputs, but it increases implementation complexity, runtime and validation requirements. Inpainting is appropriate only for small regions where surrounding pixels provide reliable context. Large missing facial regions or historical writing should not be reconstructed automatically. Any inpainted area must be marked in a restoration log and the untouched original retained.

Required implementation and evidence

Implement RETAIN, MODIFY and REDESIGN on the same minimum set of 15 colour photographs. For each strategy, save a full-resolution result and a consistent enlarged crop. Record colour correction, detail enhancement, noise reduction, damage removal and preservation of faces and background. Where no clean reference exists, do not report reference-based PSNR or SSIM.

Decision record

Complete the final decision table after executing the strategies. The choice must be supported by observed outputs and measurements. This report recommends MODIFY on engineering grounds, but the final submitted decision should be checked against the actual results obtained on the 15-image dataset.

Evaluation item	RETAIN	MODIFY	REDESIGN
Colour fidelity	Enter result	Enter result	Enter result
Damage-mask Dice	Enter result	Enter result	Enter result
Detail preservation	Enter result	Enter result	Enter result
Per-photo runtime	Enter result	Enter result	Enter result
Operational complexity	Low	Moderate	High

At least 15 images are required by the brief. Replace each entry with measured evidence and state the evaluation method.

TASK 10 – RESTORATION SYSTEM ADAPTATION (4/6): COMPARISON PROTOCOL FOR 15 COLOUR PHOTOGRAPHS

Select at least 15 colour photographs representing different exposure, colour-cast, damage and compression conditions. Run all three strategies on the same 15 images. Record dimensions, file type, input colour space, preprocessing, parameter values and processing time. Use an undamaged reference where available to calculate PSNR and SSIM. For damage masks, calculate Dice against manually annotated masks. Colour fidelity can be assessed with a colour difference measure when a reliable reference exists; otherwise use calibrated visual inspection and clearly describe its limitations. Edge/detail preservation should be assessed on fixed crops.

Required implementation and evidence

Implement RETAIN, MODIFY and REDESIGN on the same minimum set of 15 colour photographs. For each strategy, save a full-resolution result and a consistent enlarged crop. Record colour correction, detail enhancement, noise reduction, damage removal and preservation of faces and background. Where no clean reference exists, do not report reference-based PSNR or SSIM.

Decision record

Complete the final decision table after executing the strategies. The choice must be supported by observed outputs and measurements. This report recommends MODIFY on engineering grounds, but the final submitted decision should be checked against the actual results obtained on the 15-image dataset.

Evaluation item	RETAIN	MODIFY	REDESIGN
Colour fidelity	Enter result	Enter result	Enter result
Damage-mask Dice	Enter result	Enter result	Enter result
Detail preservation	Enter result	Enter result	Enter result
Per-photo runtime	Enter result	Enter result	Enter result
Operational complexity	Low	Moderate	High

At least 15 images are required by the brief. Replace each entry with measured evidence and state the evaluation method.

TASK 10 – RESTORATION SYSTEM ADAPTATION (5/6): DECISION: MODIFY AS THE PRACTICAL DEFAULT

The recommended decision is MODIFY. Retaining the grayscale pipeline without adaptation would ignore colour fidelity and high-resolution constraints. A full redesign may be justified for a future archive system, but it is not necessary as the first response to the changed input conditions. A modified pipeline can preserve the reliable classical stages while adding luminance/chrominance separation, colour-aware correction, scale-aware processing, JPEG artefact checks and cautious mask review. This decision is a reasoned design recommendation, not a claim that the three strategies have been experimentally benchmarked in this report.

Required implementation and evidence

Implement RETAIN, MODIFY and REDESIGN on the same minimum set of 15 colour photographs. For each strategy, save a full-resolution result and a consistent enlarged crop. Record colour correction, detail enhancement, noise reduction, damage removal and preservation of faces and background. Where no clean reference exists, do not report reference-based PSNR or SSIM.

Decision record

Complete the final decision table after executing the strategies. The choice must be supported by observed outputs and measurements. This report recommends MODIFY on engineering grounds, but the final submitted decision should be checked against the actual results obtained on the 15-image dataset.

Evaluation item	RETAIN	MODIFY	REDESIGN
Colour fidelity	Enter result	Enter result	Enter result
Damage-mask Dice	Enter result	Enter result	Enter result
Detail preservation	Enter result	Enter result	Enter result
Per-photo runtime	Enter result	Enter result	Enter result
Operational complexity	Low	Moderate	High

At least 15 images are required by the brief. Replace each entry with measured evidence and state the evaluation method.

TASK 10 – RESTORATION SYSTEM ADAPTATION (6/6): FAILURE CONDITION, MODEL SUPPORT AND SAFEGUARDS

The modified strategy may fail when a severe colour cast is spatially non-uniform and overlaps with faded skin tones, or when damage covers a large semantically important region. A restoration or inpainting model can assist with denoising, deblurring or small-hole completion, but generative models may invent details absent from the original. Model outputs must be stored as derived versions, not replacements. The archive should retain provenance: original file, processing configuration, mask, repaired output, reviewer decision and a record of all generated or interpolated pixels.

Required implementation and evidence

Implement RETAIN, MODIFY and REDESIGN on the same minimum set of 15 colour photographs. For each strategy, save a full-resolution result and a consistent enlarged crop. Record colour correction, detail enhancement, noise reduction, damage removal and preservation of faces and background. Where no clean reference exists, do not report reference-based PSNR or SSIM.

Decision record

Complete the final decision table after executing the strategies. The choice must be supported by observed outputs and measurements. This report recommends MODIFY on engineering grounds, but the final submitted decision should be checked against the actual results obtained on the 15-image dataset.

Evaluation item	RETAIN	MODIFY	REDESIGN
Colour fidelity	Enter result	Enter result	Enter result
Damage-mask Dice	Enter result	Enter result	Enter result
Detail preservation	Enter result	Enter result	Enter result
Per-photo runtime	Enter result	Enter result	Enter result
Operational complexity	Low	Moderate	High

At least 15 images are required by the brief. Replace each entry with measured evidence and state the evaluation method.

4. OVERALL CONCLUSION AND TECHNIQUE SELECTION

The ten tasks demonstrate that old-photograph restoration is a multi-stage decision problem rather than a single enhancement operation. Each candidate technique addresses a different degradation pattern, and the most suitable method depends on the image, damage type and preservation requirement. The final system should be conservative, traceable and reversible.

Grey-level and histogram processing

Piecewise-linear contrast stretching provides explicit control over faded tonal ranges, while CLAHE is a strong candidate for local contrast correction under uneven illumination. Gamma correction is useful for simple global tonal adjustment but a single value may not suit both bright and dark photographs. Histogram matching is valuable when a trustworthy archive reference is available.

Noise reduction and sharpening

Median filtering is appropriate for isolated impulse noise, while mild Gaussian smoothing is more suitable for approximately Gaussian grain. Large kernels can remove important detail. Unsharp masking or restrained Laplacian sharpening can improve clarity after denoising, but aggressive sharpening amplifies scratches and produces halos. Variance of Laplacian must be interpreted alongside noise and visual evidence.

Frequency-domain processing

Gaussian low-pass filtering offers a smooth frequency transition and generally avoids the ringing associated with an ideal cutoff. Homomorphic filtering is a reasonable approach to uneven illumination because it models illumination and reflectance separately. Frequency-domain outputs should be evaluated for edge loss, ringing and tone changes.

Detection and segmentation

Canny and gradient operators provide useful edge candidates, while point and directional line masks target specific damage geometries. Adaptive thresholding can handle local illumination changes better than a single global threshold. Region growing, splitting/merging and edge linking convert pixel responses into larger damage regions, but all require parameter selection and validation against manually annotated masks.

Final system recommendation

Use a modified, luminance-aware pipeline for colour and high-resolution photographs. Preserve the original, create a working copy, correct illumination and contrast conservatively, remove limited noise, detect damage, segment candidate regions, request review for ambiguous areas and repair only small confirmed defects. Store masks and provenance with the output.

Stage	Recommended candidate	Reason
Tone correction	Piecewise stretch / CLAHE	Controlled global and local contrast.
Impulse noise	Median filter	Suppresses isolated extreme pixels.

Mild Gaussian noise	Gaussian smoothing	Weighted smoothing with limited complexity.
Sharpening	Unsharp masking	Adjustable and easy to constrain.
Low-pass filtering	Gaussian LPF	Smooth transition with reduced ringing.
Illumination correction	Homomorphic filtering	Explicit illumination/reflectance model.
Edge candidates	Canny + directional masks	Thin edges plus geometry-specific responses.
Damage mask	Adaptive threshold + region refinement	Local response under uneven illumination.
Colour adaptation	Luminance/chrominance separation	Reduces unintended colour shifts.

5. LIMITATIONS, ETHICAL PRESERVATION AND FUTURE WORK

A technically attractive restoration can still be historically misleading if it changes a person's facial features, erases handwriting or invents missing objects. The system must make the distinction between enhancement and reconstruction explicit. The original scan is the primary record; every processed version is a derivative.

Limitations of classical methods

Classical filters rely on local intensity, frequency or geometric assumptions. A stain and a dark garment may have similar intensity. A crease can resemble a real edge. A scratch may cross a face and be difficult to distinguish from a hair strand. Global transformations do not understand image content, while local methods may amplify noise or create patchy contrast. Parameters selected for one photograph may not generalize to another.

Limitations of learned models

Pretrained models may require GPU resources, memory, input resizing or model-specific preprocessing. Their training data may not represent historical photographs. Generative inpainting may create plausible but unsupported facial details, lettering or background objects. A model's confidence score does not establish historical correctness. Learned outputs should be evaluated on the same images and annotations as classical methods.

Preservation and audit trail

Maintain an untouched source file, checksum or stable identifier, processing configuration, software version, output image, damage mask and review notes. Keep repaired pixels distinguishable from directly observed pixels. Do not erase handwritten dates or notes merely because they appear irregular. If the back of a photograph contains writing, process it as a separate documentary image.

Future improvements

Future work may include a larger annotated archive dataset, damage-type-specific segmentation, colour calibration targets, scanner stripe correction, model-assisted review, uncertainty estimation and a human-in-the-loop interface. A deployment version should profile memory use and runtime on the exact four-core CPU environment. A quality gate can reject outputs with excessive clipping, colour shift or loss of edge detail.

- Never overwrite the original photograph.
- Do not claim that inpainted details are authentic historical evidence.
- Record all parameters and measured performance.
- Use human review for faces, handwriting and large damaged regions.
- Report failure cases as carefully as successful examples.

APPENDIX A – REUSABLE PYTHON / OPENCV IMPLEMENTATION TEMPLATE

The following compact template provides a consistent starting point for experiments. Extend it for each candidate method and record parameters in a structured log. The template does not claim to have been executed on a supplied photograph; it must be run against the selected dataset.

A.1 Loading and validation

Use cv2.imread with an explicit mode and verify that the returned array is not None. Record image height, width, channel count and dtype. For grayscale tasks, use cv2.IMREAD_GRAYSCALE. For colour tasks, use cv2.IMREAD_COLOR and remember that OpenCV uses BGR channel ordering.

A.2 Example code

The code below illustrates safe loading, several transformations, smoothing, thresholding and quality metrics. Add separate functions for Fourier filters, edge detectors, region growing and colour correction. Do not use the same output filename for different parameter settings.

A.3 Reproducibility

Store a CSV row for each run with image ID, method, parameters, runtime, output path and metric values. Run each method several times and report the median or mean runtime with the number of repetitions. Record the CPU, RAM, Python, OpenCV and NumPy versions.

Code listing follows on the next page.

APPENDIX A – CODE LISTING

```
import cv2
import numpy as np
import time

def load_gray(path):
    image = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if image is None:
        raise FileNotFoundError(path)
    return image

def gamma_transform(image, gamma=0.8):
    x = image.astype(np.float32) / 255.0
    y = np.power(x, gamma)
    return np.clip(y * 255.0, 0, 255).astype(np.uint8)

def piecewise_stretch(image, low=40, high=210):
    x = image.astype(np.float32)
    if high <= low:
        raise ValueError("high must be greater than low")
    y = (x - low) * (255.0 / (high - low))
    return np.clip(y, 0, 255).astype(np.uint8)

def clahe_enhance(image, clip=2.0, grid=(8, 8)):
    op = cv2.createCLAHE(clipLimit=clip, tileGridSize=grid)
    return op.apply(image)

def filter_candidates(image, k=3):
    return {
        "box": cv2.blur(image, (k, k)),
        "gaussian": cv2.GaussianBlur(image, (k, k), 0),
        "median": cv2.medianBlur(image, k)
    }

def threshold_candidates(image, block=31, C=5):
    _, otsu = cv2.threshold(
        image, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    adaptive = cv2.adaptiveThreshold(
        image, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY, block, C)
    _, fixed = cv2.threshold(image, 128, 255, cv2.THRESH_BINARY)
    return {"fixed": fixed, "otsu": otsu, "adaptive": adaptive}
```

```
def dice_score(pred, truth):
    p = pred > 0
    t = truth > 0
    denom = p.sum() + t.sum()
    if denom == 0:
        return 1.0
    return 2.0 * np.logical_and(p, t).sum() / denom

def iou_score(pred, truth):
    p = pred > 0
    t = truth > 0
    union = np.logical_or(p, t).sum()
    if union == 0:
        return 1.0
    return np.logical_and(p, t).sum() / union

def timed_call(function, *args, repeats=5, **kwargs):
    samples = []
    result = None
    for _ in range(repeats):
        start = time.perf_counter()
        result = function(*args, **kwargs)
        samples.append((time.perf_counter() - start) * 1000.0)
    return result, float(np.median(samples))

if __name__ == "__main__":
    image = load_gray("input_photo.png")
    enhanced, ms = timed_call(gamma_transform, image, 0.8)
    cv2.imwrite("output_gamma.png", enhanced)
    print("Gamma runtime (median ms):", ms)
```

APPENDIX B – EXPERIMENTAL RESULT RECORDING SHEETS

Complete these tables after running the implementation. The assignment brief specifically requires the same photographs to be used for candidate comparisons and prohibits assumed or fabricated measurements. Keep one row per image and parameter set; report summary statistics only after the raw observations are available.

B.1 Damage annotation checklist

For each photograph, annotate dust spots, scratches, stains, creases, tears and missing areas separately where possible. Use a consistent binary mask convention and preserve the original annotation. If the same pixel belongs to more than one damage category, define how overlap is handled before calculating metrics.

B.2 Runtime measurement

Measure each stage independently and the complete pipeline end-to-end. Include image loading and saving only if the assignment's timing interpretation includes them; state the choice. Use the same machine, image dimensions and warm-up policy for all candidates. Report median runtime and range across repeated runs.